

24. live 自動校正ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/solar_autocal_node.py
種別	Python
区分	runtime node
役割	観測 solar/pack power と planner 予測との差から solar_gain、drive_power_gain、aux_power_w を推定 publish する。
起動文脈	運用中に mpc_node の係数を微調整する補助ノード。

このファイルを読む理由

観測 solar/pack power と planner 予測との差から solar_gain、drive_power_gain、aux_power_w を推定 publish する。

- /calib/* topic を publish する。
- mpc_node がそれを購読して内部 gain を更新する。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/live_launch.py

次に読むと理解が進むファイル

- mpc_solarcar/solar_autocal_logic.py
- mpc_solarcar/mpc_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_autocal_logic.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
return max(float(lo), min(float(hi), float(value)))

class SolarAutocalNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_autocal_node")
        self.declare_parameter("publish_period_sec", 30.0)
        self.declare_parameter("stationary_speed_kmh", 2.0)
        self.declare_parameter("drive_speed_kmh", 25.0)
        self.declare_parameter("day_ghi_threshold", 150.0)
        self.declare_parameter("alpha", 0.2)
        self.declare_parameter("solar_gain_init", 1.0)
        self.declare_parameter("drive_power_gain_init", 1.0)
        self.declare_parameter("aux_power_w_init", 8.0)
        self.declare_parameter("solar_gain_min", 0.5)
        self.declare_parameter("solar_gain_max", 1.5)
        self.declare_parameter("drive_power_gain_min", 0.7)
        self.declare_parameter("drive_power_gain_max", 1.4)
        self.declare_parameter("aux_power_w_min", 0.0)
        self.declare_parameter("aux_power_w_max", 300.0)

        self.period = max(1.0, float(self.get_parameter("publish_period_sec").value))
```

主要構造の読み取り

主要クラスは SolarAutocalNode。主要関数は main。ROS パラメータ宣言は 14 件。ROS I/O は publisher 4 件、subscription 6 件。

主要クラス・関数

行	種別	名前	補足
17	class	SolarAutocalNode	bases: Node
13	function	_clamp	args: value, lo, hi
18	function	__init__	args: self
74	function	_on_speed	args: self, msg
77	function	_on_current	args: self, msg
82	function	_on_voltage	args: self, msg
87	function	_on_solar	args: self, msg
90	function	_on_env	args: self, msg
95	function	_on_metrics	args: self, msg
102	function	_ema	args: self, current, estimate, lo, hi
106	function	_step	args: self
153	function	main	

ファイルを上から読んだときの定義順

行	内容
13	関数 _clamp を定義する。

- 17 クラス SolarAutocalNode を定義する。
- 153 関数 main を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importmath</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L50, L51, L52, L53, L54, L55, L69, L70, ...。
5	<code>importrcipy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L154, L156, L158。
6	<code>fromrcipy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L17。
8	<code>fromstd_msgs. msgimportFloat32, Float32MultiArray,String</code>	Float32 や String など軽量メッセージで <code>planner/vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L57, L58, L59, L60, L62, L63, L64, L65, ...。
10	<code>from.solar_autocal_ logicimportdaytime_ stationary_aux_estimate</code>	自動校正ロジック関数群から <code>daytime_stationary_aux_estimate</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_autocal_logic.py</code> 。このファイル内での主な使用位置は L110。

関数・クラスを上から順に解説

L13 関数 _clamp

- 定義: `_clamp(value, lo, hi)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`, `min`
- 戻り値の要点: `max(float(lo), min(float(hi), float(value)))`

1. `max(float(lo), min(float(hi), float(value)))` を返す。

L17 クラス SolarAutocalNode

- 定義: `SolarAutocalNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `__init__`, `_clamp`, `_ema`, `abs`, `create_publisher`, `create_subscription`, `create_timer`, `daytime_stationary_aux_estimate`, `declare_parameter`, `float`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `_on_speed` を定義する。

3. 関数 `_on_current` を定義する。
4. 関数 `_on_voltage` を定義する。
5. 関数 `_on_solar` を定義する。
6. 関数 `_on_env` を定義する。
7. 関数 `_on_metrics` を定義する。
8. 関数 `_ema` を定義する。
9. 関数 `_step` を定義する。

L153 関数 `main`

- 定義: `main()`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `SolarAutocalNode`, `destroy_node`, `init`, `shutdown`, `spin`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `rcipy.init(...)` を実行する。
 2. `node` に `SolarAutocalNode()` の結果を代入する。
 3. `rcipy.spin(...)` を実行する。
 4. `node.destroy_node(...)` を実行する。
 5. `rcipy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
20	<code>publish_period_sec</code>	30.0
21	<code>stationary_speed_kmh</code>	2.0
22	<code>drive_speed_kmh</code>	25.0
23	<code>day_ghi_threshold</code>	150.0
24	<code>alpha</code>	0.2
25	<code>solar_gain_init</code>	1.0
26	<code>drive_power_gain_init</code>	1.0
27	<code>aux_power_w_init</code>	8.0
28	<code>solar_gain_min</code>	0.5
29	<code>solar_gain_max</code>	1.5
30	<code>drive_power_gain_min</code>	0.7
31	<code>drive_power_gain_max</code>	1.4
32	<code>aux_power_w_min</code>	0.0
33	<code>aux_power_w_max</code>	300.0

ROS topic I/O

行	種別	topic	callback / 補足
57	publisher	/calib/solar_gain	publisher
58	publisher	/calib/drive_power_gain	publisher
59	publisher	/calib/aux_power_w	publisher
60	publisher	/calib/status	publisher
62	subscription	/vehicle/speed_kmh	self._on_speed
63	subscription	/vehicle/batt_current_a	self._on_current
64	subscription	/vehicle/batt_voltage_v	self._on_voltage
65	subscription	/vehicle/solar_power_w	self._on_solar
66	subscription	/planner/env	self._on_env
67	subscription	/planner/metrics	self._on_metrics

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_autocal_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。